Open in app ↗ Sign up Sign In

Published in Towards Data Science

Thomas Albin Follow

Apr 21, 2020 · 10 min read · ✨ · Listen

Save

HTTPS://TOWARDSDATASCIENCE.COM/TAGGED/SPACE-SCIENCE-WITH-PYTHON

Space Science with Python — Setup and first steps

A starting guide to became a Citizen Space Scientist — Part 1

Photo by SpaceX on Unsplash

133 | 5

Space Science with Python. It may sound challenging, complex and broad. During my time in the university, I got some experience in different Space Science related fields that I linked to data science and machine learning methods. With this part #1 of my tutorial series, I would like to share my experience and knowledge with the community: with all the developers, makers, students or passionate space and programming fans out there.

I provide a lot of details in the beginning so that everyone is able to follow the tutorial and the content of the library. After some time, when we get more familiar with everything, we can focus on scientific tasks beyond the basics.

Prerequisites

To follow and understand the tutorials I recommend having a solid basic knowledge of Python. We will work with Python 3 and I recommend setting up a virtual environment on your machine, where you can install all necessary packages and where you can "play around". Below, you will find a brief description of how to set up a virtual environment. All scripts are uploaded on my public GitHub (https://github.com/ThomasAlbin/SpaceScienceTutorial) repository where you can find the Jupyter notebooks and stand-alone Python scripts. I would recommend cloning the repository. The installation of all shown libraries has been tested on Linux (Ubuntu and Debian) as well as on Mac OS. If you encounter any issues on Windows, let's try to figure it out together, for example on the corresponding Reddit post that brought you here.

Some basic knowledge of the Solar System and its celestial bodies is a small advantage, but again: do not worry. We start with basic concepts, and between the tutorials, you will have some time to read into these topics. If you would like to have more explanations in this regard, please let me know.

Python setup

First, we need to set up a virtual environment where we can install our packages. Open the terminal and change to the directory, where you want to install your environment. Assuming that you already installed Python 3, you can set up the

environment by executing the following command. *pythonenv* will be the name of the environment:

```
python3 -m venv pythonenv
```

You can now activate the virtual environment by executing:

```
source pythonenv/bin/activate
```

In the terminal, next to your username, the name of the environment should be displayed in round brackets like *(pythonenv).*

With the Python package manager *pip3* one can easily install any Python library. For starters, I would recommend installing the SciPy suite that will be used frequently. It also contains Jupyter (it can be started within the cloned Git project by executing *jupyter notebook*):

```
pip3 install numpy scipy matplotlib ipython jupyter pandas sympy
nose
```

SPICE installation

In the next couple of weeks, we will focus together on the Solar System, its bodies, spacecraft missions and scientific results. We will learn miscellaneous methods, data scientific approaches and libraries. On my very first post, I mentioned a NASA toolkit called SPICE (*Spacecraft Planet Instrument C-matrix Events*). This toolkit will be the core of our first sessions. SPICE is a huge toolkit that is being used by the Solar System science community to determine e.g., a planet's position, the coordinates of an asteroid in the sky, or whether the Field-Of-View of a spacecraft camera aims to the surface of a celestial body (and much more).

NASA provides a sophisticated overview with documentation and presentations on their website: SPICE NAIF. The toolkit is provided in C, Fortran, IDL, Matlab and Java. Thanks to the community member *AndrewAnnex*, a Python wrapper is

available that uses the C version of SPICE: https://github.com/AndrewAnnex/SpiceyPy.

You can install the Python wrapper by executing the following command. However, please ensure that you have a C compiler installed like gcc on Linux. XCode should provide all necessary software requirements for your Mac system:

```
pip3 install spiceypy
```

Now we are ready for lift-off.

SPICE Introduction

Before we dive into the programming part, let's have a look at SPICE. SPICE has been developed to provide a common toolkit for planetary scientists and engineers to compute miscellaneous space mission-relevant information like:

- Reference frames
- Positions and velocities
- Orientation/Pointing
- Size/Shapes/Physical parameters
- Time conversion

We will aim all these topics to understand what they mean and where to use them.

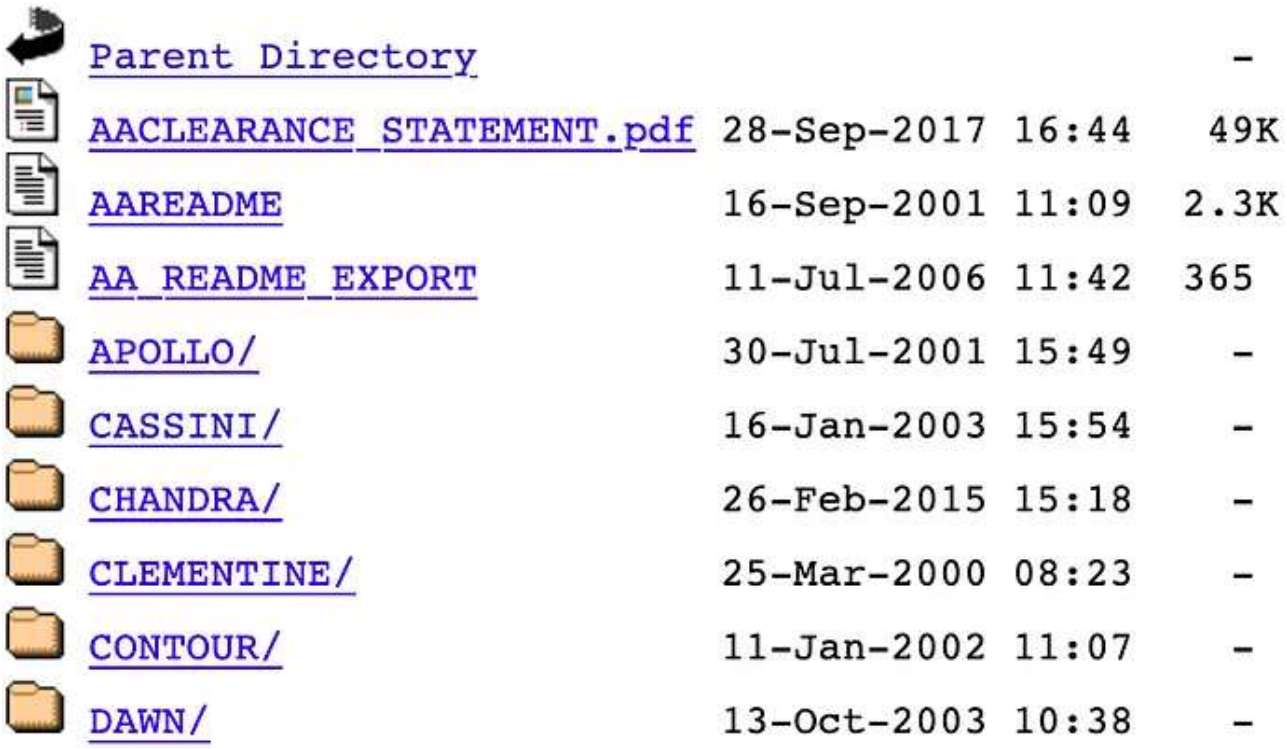
SPICE does not calculate computational heavy tasks like N-body simulations, re-entry scenarios or complex manoeuvrers. It uses existing data to provide common access to a large variety of Solar System related problems (a short overview that is provided by NASA can be seen here: NASA Spice Overview).

Out of the box, SPICE cannot calculate a lot. It requires auxiliary data, so-called *kernels* to work properly. These kernels are separated into different categories, like:

- **spk** contain trajectory information of planetary bodies, spacecraft, etc.
- **pck** contain physical parameters of bodies like the size or orientation

- **ik** contain instrument-specific information that are e.g., mounted on a spacecraft
- **ck** contain information regarding the orientation of a spacecraft in space
- **fk** contain reference frame information that is needed to calculate positions in a less common reference system
- **lsk** contain time information that is crucial to convert e.g., the UTC time into ephemeris time ET (a standard time format that is being used in space science and astronomy)

We will get in touch with all of these kernels. You may ask: Why are the information stored in additional files? Why is the data not stored in one huge toolkit, since we should *know* the trajectory of certain spacecraft? Well, you are right, but there are dozens of missions out there and a lot of missions are being prepared. Especially the planned missions have “pre-planned” or “predicted” trajectories that are frequently updated. Further, different institutes work on different missions. It would be a struggle to contact everyone to gather all information. Thus, each mission, instrument and so on is separated. NASA and ESA provide repositories with a lot of kernels. One example is here: SPICE Kernels



Parent Directory			-
AACLEARANCE_STATEMENT.pdf	28-Sep-2017	16:44	49K
AAREADME	16-Sep-2001	11:09	2.3K
AA_README_EXPORT	11-Jul-2006	11:42	365
APOLLO/	30-Jul-2001	15:49	-
CASSINI/	16-Jan-2003	15:54	-
CHANDRA/	26-Feb-2015	15:18	-
CLEMENTINE/	25-Mar-2000	08:23	-
CONTOUR/	11-Jan-2002	11:07	-
DAWN/	13-Oct-2003	10:38	-

The NASA / NAIF kernel repository (only a small fraction)

The screenshot above shows some mission kernel folders, like the Apollo missions or the Cassini spacecraft that went to Saturn. A core part is stored in the *generic_kernels* folders, where planetary information are stored. For starters, all kernels that are needed for this article are stored on my GitHub repository. Later, however, we will need kernels that are huge and need to be downloaded and manually stored by you. We will get to this point in a few weeks. As you can see in the downloaded repository, I prepared a kernel folder called *_kernels*. We will use the kernels in a few moments.

The folder structure of the Space Science with Python tutorial. You can see that some kernels have been downloaded and prepared.

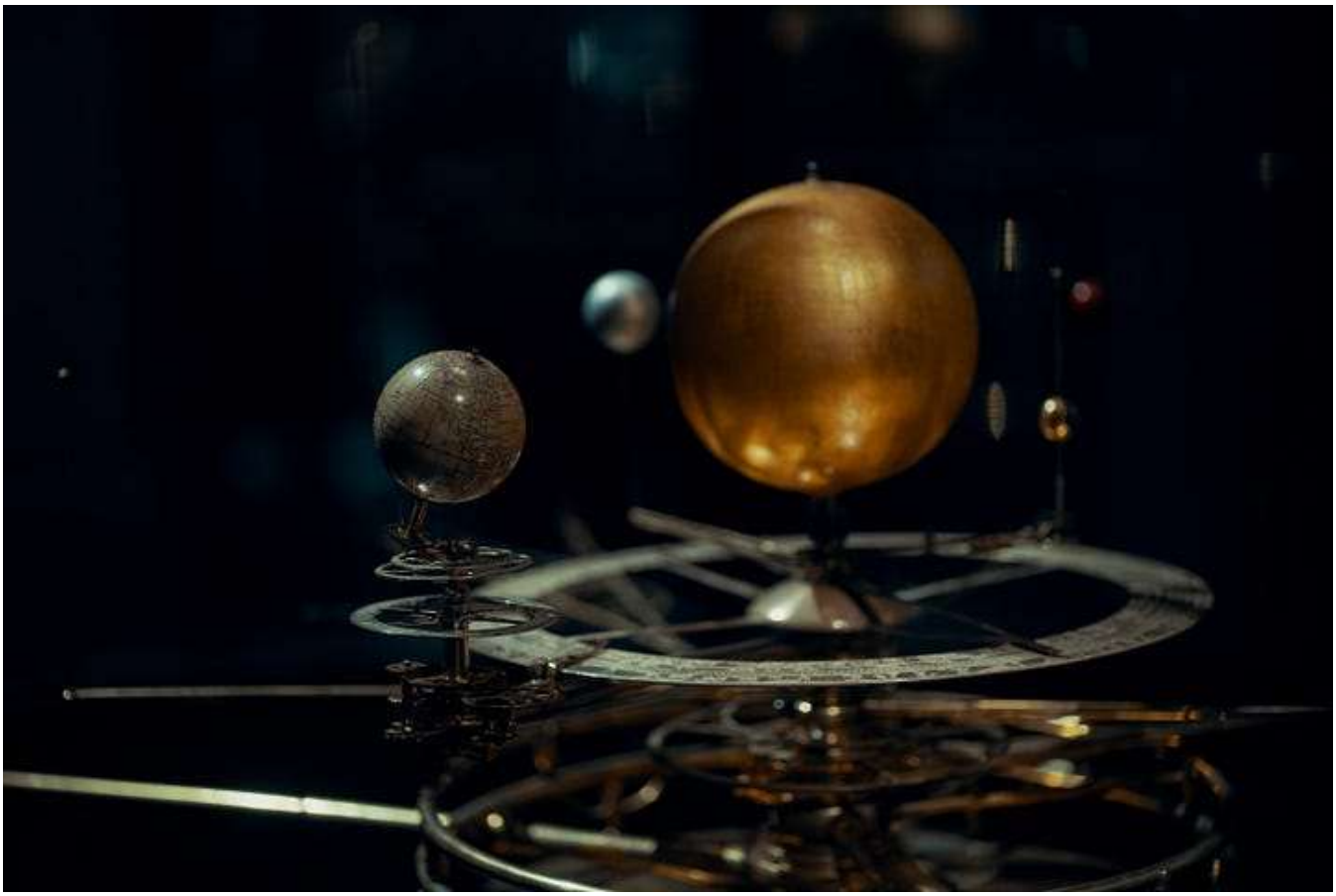


Photo by Ross Sneddon on Unsplash

Our first steps

Now that we are set up, we can define the first goal for today. Let's compute the Earth's position and velocity vector (so-called state vector) for today, midnight. Further, we compare the orbital speed of our home planet with the theoretical expectation.

First, we import the SPICE wrapper library *spicepy*. If no error occurs the installation went well and we can continue.

Part 1/13

We want to determine the position of our home planet with respect to the Sun. How can we achieve this with SPICE? Let's have a look at the reference guide of SPICE: SPICE Docs.

Main Page

Index of CSPICE functions

A B C D E F G H I J K L M N O P Q R S T U V W X

A

- APPNDC_C - Append an item to a character cell
- APPNDD_C - Append an item to a double precision cell
- APPNDI_C - Append an item to an integer cell
- AXISAR_C - Axis and angle to rotation

B Top

- B1900_C - Besselian Date 1900.0
- B1950_C - Besselian Date 1950.0
- BADKPV_C - Bad Kernel Pool Variable
- BLTFRM_C - Built-in frame IDs
- BODC2N_C - Body ID code to name translation
- BODC2S_C - Body ID code to string translation
- BODDEF_C - Body name/ID code definition
- BODFND_C - Find values from the kernel pool
- BODN2C_C - Body name to ID code translation
- BODS2C_C - Body string to ID code translation
- BODVAR_C - Return values from the kernel pool
- BODVCD_C - Return d.p. values from the kernel pool
- BODVRD_C - Return d.p. values from the kernel pool
- BRCKTD_C - Bracket a d.p. value within an interval
- BRCKTI_C - Bracket an integer value within an interval
- BSCHOC_C - Binary search with order vector, character
- BSCHOI_C - Binary search with order vector, integer
- BSRCHC_C - Binary search for a character string
- BSRCHD_C - Binary search for a double precision value
- BSRCHI_C - Binary search for an integer value

C Top

- CARD_C - Cardinality of a cell
- CCIFRM_C - Class and class ID to associated frame
- CGV2EL_C - Center and generating vectors to ellipse
- CHBDER_C - Derivatives of a Chebyshev expansion
- CHKIN_C - module Check In
- CHKOUT_C - Module Check Out

Screenshot of the SPICE reference guide

Well, there are a lot of functions (and do not forget that we need kernels, too). It appears to be overwhelming, but that's what the tutorial is for: to support you in this regard.

What we need is the function *spkgeo* (Please note: this is the documentation of the C library, thus every function name has the *_C* suffix that needs to be removed for Python): spkgeo doc.

The documentation says:

> *Abstract*

> *Compute the geometric state (position and velocity) of a target body relative to an observing body.*

That's what we need. The input parameters are the target body (Earth), the ET, the reference frame and the observer (Sun).

So in a first step, we need to convert a UTC string to the ET. We use the function *utc2et* for this purpose: utc2et doc.

We determine today's date and create a string of the format *year-month-day-hours:minutes:seconds*. The documentation of the *utc2et* function provides all possible time formats in the section *Examples*. Although it is not required to explicitly write 00:00:00 for midnight, we do it for the sake of completeness (Spoiler: the following code snippet causes an error!):

Part 2/13

An error occurred. Why? As mentioned before, most information is stored in the kernel files. Here, we cannot convert between the times, because of a missing lsk. The tutorial repository contains the needed file (*naif0012.tls*) and can be found in the official repository here: SPICE lsk kernels. The kernel is loaded with the SPICE function *furnsh*.

Part 3/13

We re-execute the command…

Part 4/13

… and no error occurs. Let's have a look at the value:

Part 5/13

For today (2020.04.21) the result is 640699269.1855832.

Now let's try the command *spkgeo* that shall compute the Earth's state vector. But what are the target's and observer's name? Apparently "Earth" and "Sun"; however, SPICE uses the so-called NAIF IDs to identify an object. The complete list can be found here: NAIF IDs.

Search for the number 399. That's the Earth! And the Sun has the number 10. The reference frame is set as "ECLIPJ2000" and refers to the ecliptic reference frame at the time J2000 (no kernel needed for this basic frame). Image a table and at the centre of the table, you put something that represents the Sun. Use another object representing the Earth and circle it around the Sun. The table is the so-called "Ecliptic Plane" of our planet. Other planets have their own plane and are slightly inclined with respect to the Earth's plane. Why J2000? Astronomers set the plane for our planet at a certain time of the year 2000. Due to gravitational perturbations, our current "2020 plane" is very, very slightly inclined with respect to the J2000 plane. Instead of re-defining it every year, we use the 2000-version (Spoiler: another error appears here (the last one, I promise!)):

Another kernel is missing. This time it's an spk. Let's have a deep dive into this problem. We go to the repository page (you do not need to download anything, since I already put it in the *_kernel* folder): SPICE kernels

... and go to *generic_kernels*. There we need to go into *spk* and then *planets* since we want to compute the state vector of our planet. We find the following status for today:

Name	Last modified	Size
Parent Directory		-
a_old_versions/	19-Mar-2018 15:48	-
aa_spk_production_dates_by_alpha.txt	07-Jan-2019 13:00	310
aa_spk_production_dates_by_date.txt	07-Jan-2019 13:01	310
aa_summaries.txt	07-Jan-2019 13:00	5.3K
aareadme_de430-de431.txt	01-Jul-2014 11:11	3.1K
aareadme_de432s.txt	30-Jun-2014 16:02	598
de430.bsp	03-Sep-2013 12:13	114M
de430_and_de431.pdf	05-Mar-2014 12:13	3.5M
de430_moon_coord.pdf	27-Nov-2013 15:19	705K
de430_tech-comments.txt	01-Jul-2014 11:12	50K
de431_part-1.bsp	03-Sep-2013 12:16	1.3G
de431_part-2.bsp	03-Sep-2013 12:43	1.7G
de431_tech-comments.txt	01-Jul-2014 11:12	50K
de432.pdf	01-May-2014 16:53	2.2M
de432_tech-comments.txt	01-Jul-2014 11:12	50K
de432s.bsp	01-May-2014 16:40	10M
de435.bsp	07-Jan-2019 12:39	114M
de435_tech-comments.txt	15-Apr-2020 16:32	29K
de438.bsp	28-Aug-2019 10:42	114M

Screenshot of: https://naif.jpl.nasa.gov/pub/naif/generic_kernels/spk/planets/

Every folder has *aa_*.txt* files. Let's check *aa_summaries.txt*. The file lists meta data for miscellaneous kernels. It shows which objects are computed w.r.t. which other body object. Note: Mercury's barycentre and Venus' barycentre are both available w.r.t. the Solar System barycentre. Thanks to simple addition SPICE can easily compute the distance between both planetary barycentres. The last line shows the

time coverage. We take the file *de432s.bsp*, since it is quite small and covers the time interval of our interest:

```
Summary for: de432s.bsp

Bodies: MERCURY BARYCENTER (1) w.r.t. SOLAR SYSTEM BARYCENTER (0)
        VENUS BARYCENTER (2) w.r.t. SOLAR SYSTEM BARYCENTER (0)
        EARTH BARYCENTER (3) w.r.t. SOLAR SYSTEM BARYCENTER (0)
        MARS BARYCENTER (4) w.r.t. SOLAR SYSTEM BARYCENTER (0)
        JUPITER BARYCENTER (5) w.r.t. SOLAR SYSTEM BARYCENTER (0)
        SATURN BARYCENTER (6) w.r.t. SOLAR SYSTEM BARYCENTER (0)
        URANUS BARYCENTER (7) w.r.t. SOLAR SYSTEM BARYCENTER (0)
        NEPTUNE BARYCENTER (8) w.r.t. SOLAR SYSTEM BARYCENTER (0)
        PLUTO BARYCENTER (9) w.r.t. SOLAR SYSTEM BARYCENTER (0)
        SUN (10) w.r.t. SOLAR SYSTEM BARYCENTER (0)
        MERCURY (199) w.r.t. MERCURY BARYCENTER (1)
        VENUS (299) w.r.t. VENUS BARYCENTER (2)
        MOON (301) w.r.t. EARTH BARYCENTER (3)
        EARTH (399) w.r.t. EARTH BARYCENTER (3)
        Start of Interval (ET)              End of Interval (ET)
        -----------------------------       -----------------------------
        1949 DEC 14 00:00:00.000            2050 JAN 02 00:00:00.000
```

Part of https://naif.jpl.nasa.gov/pub/naif/generic_kernels/spk/planets/aa_summaries.txt

We load the kernel:

Part 7/13

Re-compute the state vector:

Part 8/13

And print it out. We get: [-1.28760839e+08, -7.76110220e+07, 4.32943029e+03, 1.48939328e+01, -2.56361729e+01, 1.00712291e-03]. The first 3 values are the x, y, z components in km. The last 3 values are the corresponding velocity components in km/s.

We can verify the results by using NASA's HORIZONS Web-Interface.

In an extra tutorial, I will explain to you how to use this web application, since it allows one to adjust a lot of parameters. After some clicking I got the following results: [-1.287618689589618E+08, -7.760924828577068E+07, 4.329366157151759E+03, 1.489358282611585E+01, -2.563638289196221E+01, 1.007311359298768E-03]. We compare the x-position component and notice a deviation of around 1000 km. That is not a lot, but theoretically, it should be 0 km?! Well, in this case, HORIZONS uses a kernel called *DE431mx*. Different kernels may have different precise values.

Another simple way to check whether our results make sense is to determine the distance between the Sun and Earth. It should be around 1 astronomical unit (1 AU). First, we compute the distance in km:

Part 10/13

And with the SPICE function *convrt* (convrt doc) we can change the km values to AU (no kernel needed). The results is close to 1 AU.

Part 11/13

Let's do the last task. We compute the orbital speed of the Earth in km/s:

The value is close to 30 km/s. But does it make sense? The theoretical expectation of the orbital velocity can be approximated with the following equation, where ***G*** is the gravitational constant, ***M*** is the mass of the Sun, and ***r*** is the distance between the Earth and Sun:

$$v_{\text{orb}} \approx \sqrt{\frac{GM}{r}}$$

We use the kernel *gm_de431.tpc* from the *generic_kernels* directory (pck) and load it. With the function *bodvcd* we get the ***G*M*** value for the Sun (bodvcd doc). The *bodyid* is again 10, the required *item* is ***GM*** and *maxn* is an input parameter that sets the number of expected returns (here, it is 1):

Again, we get a value close to 30 km/s!

Conclusion

I hope you enjoyed the first tutorial and got an impression of how SPICE "thinks" and how to use it. We will use this library and explore further features in the next tutorials to get more hands-on experience. The SPICE introductions will be a solid basis for later scientific tutorials.

Please let me know (here or on Reddit: https://www.reddit.com/user/MrAstroThomas), if you liked it, what you missed or any other question. I try to answer as many as possible.

The next tutorial will be published on Saturday (2020.04.25), hopefully with a GitHub Gist integration. We will analyse the first law of Kepler's law:

> *The orbit of a planet is an ellipse with the Sun at one of the two foci*

Is this statement 100 % correct? We will see.

I wish you all the best during these challenging times,

Thomas

Python Space Science Programming Space Science With Python

Sign up for The Variable

By Towards Data Science

Every Thursday, the Variable delivers the very best of Towards Data Science: from hands-on tutorials and cutting-edge research to original features you don't want to miss. Take a look.

By signing up, you will create a Medium account if you don't already have one. Review our Privacy Policy for more information about our privacy practices.

Get this newsletter

About Help Terms Privacy

Get the Medium app

Download on the App Store

GET IT ON Google Play